

链表参数传递思考题

2351136 大数据 李盛鹏

原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

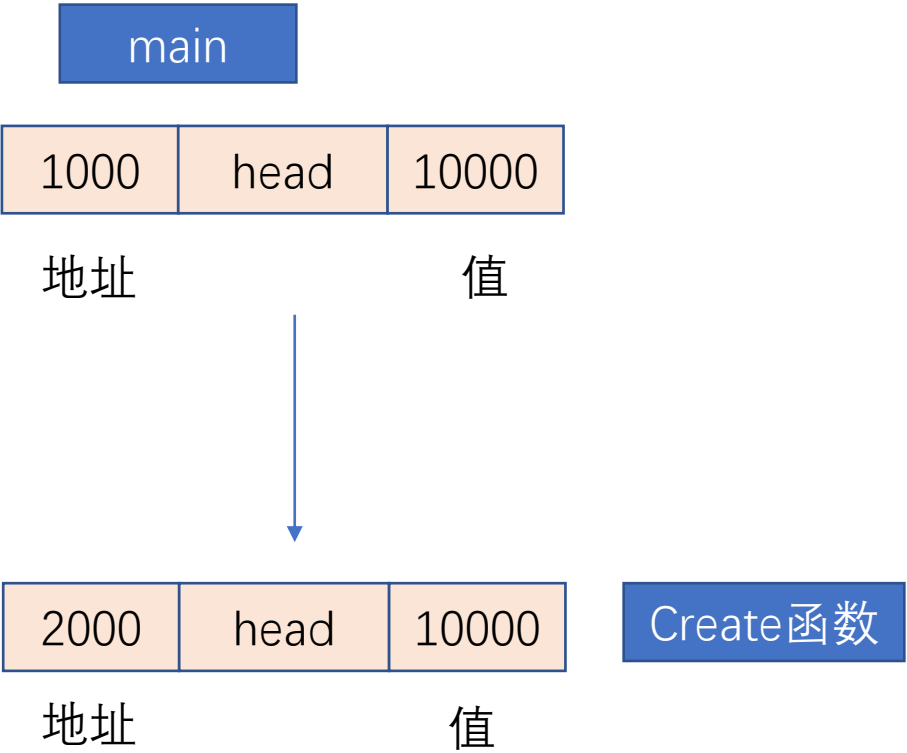
    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

链表的建立不正确, 因为 head 在此处是一个形参的地址, 尽管能建立链表, 但是不能将 main 中的 head 进行修改。

原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

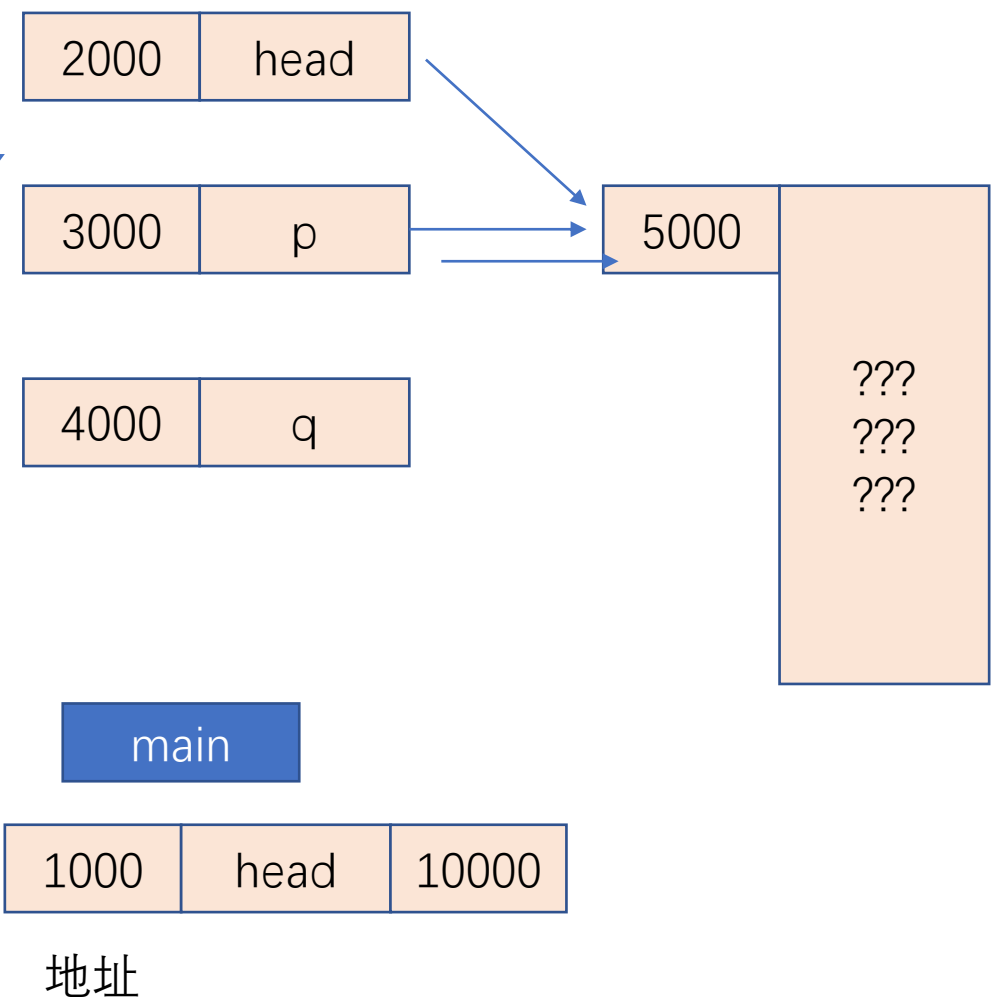


原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是第一个元素

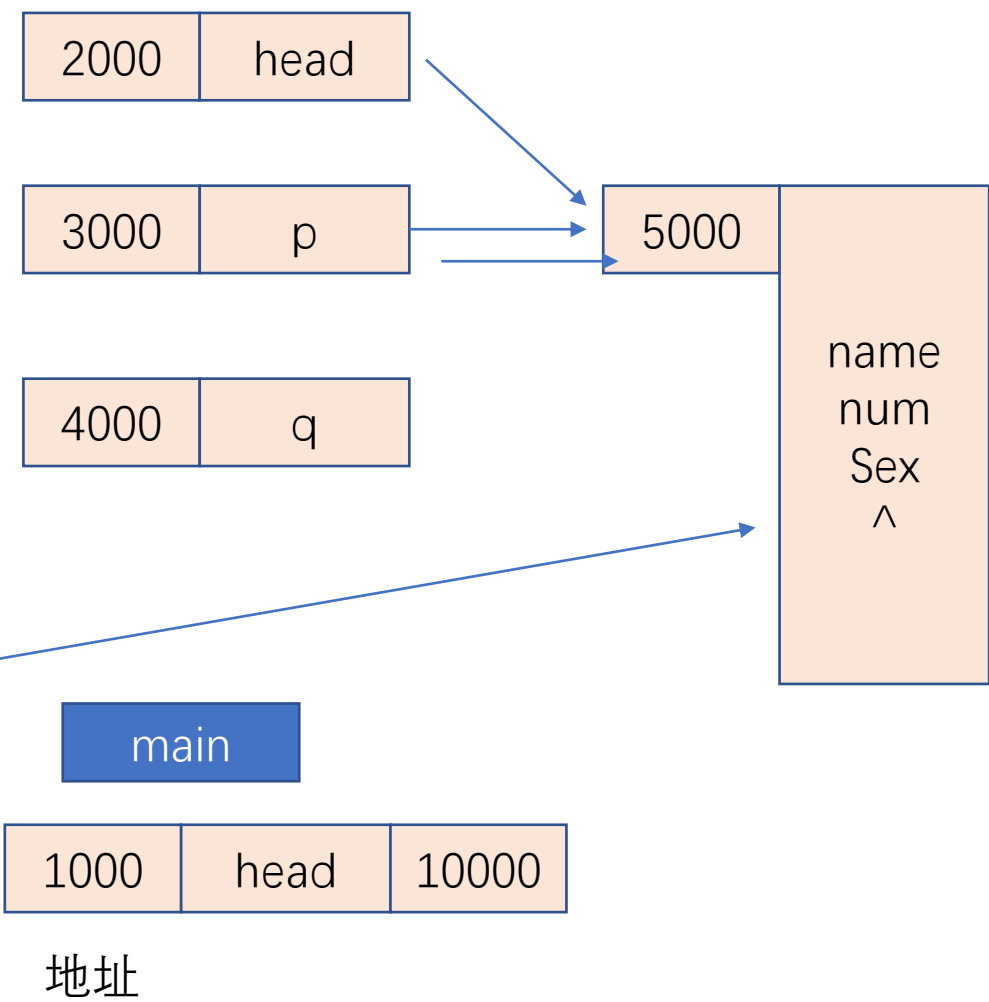


原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是第一个元素

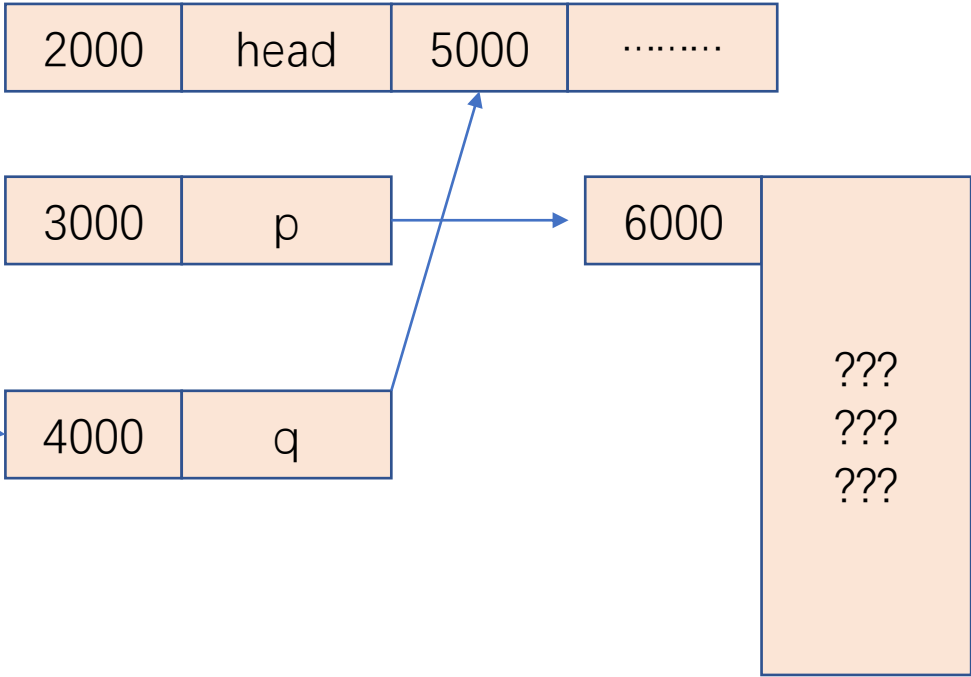


原先的create函数

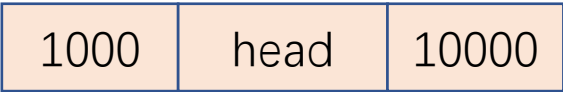
```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是后面的元素



main



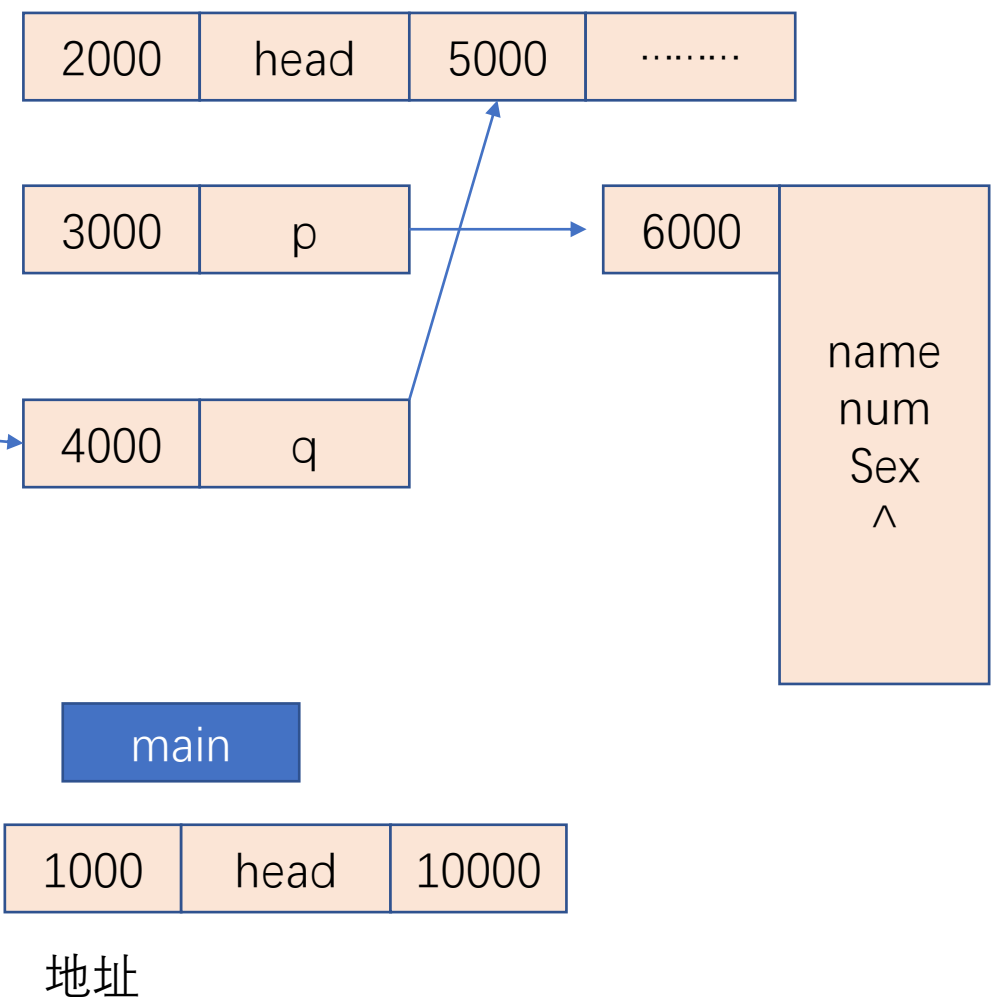
地址

原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是后面的元素

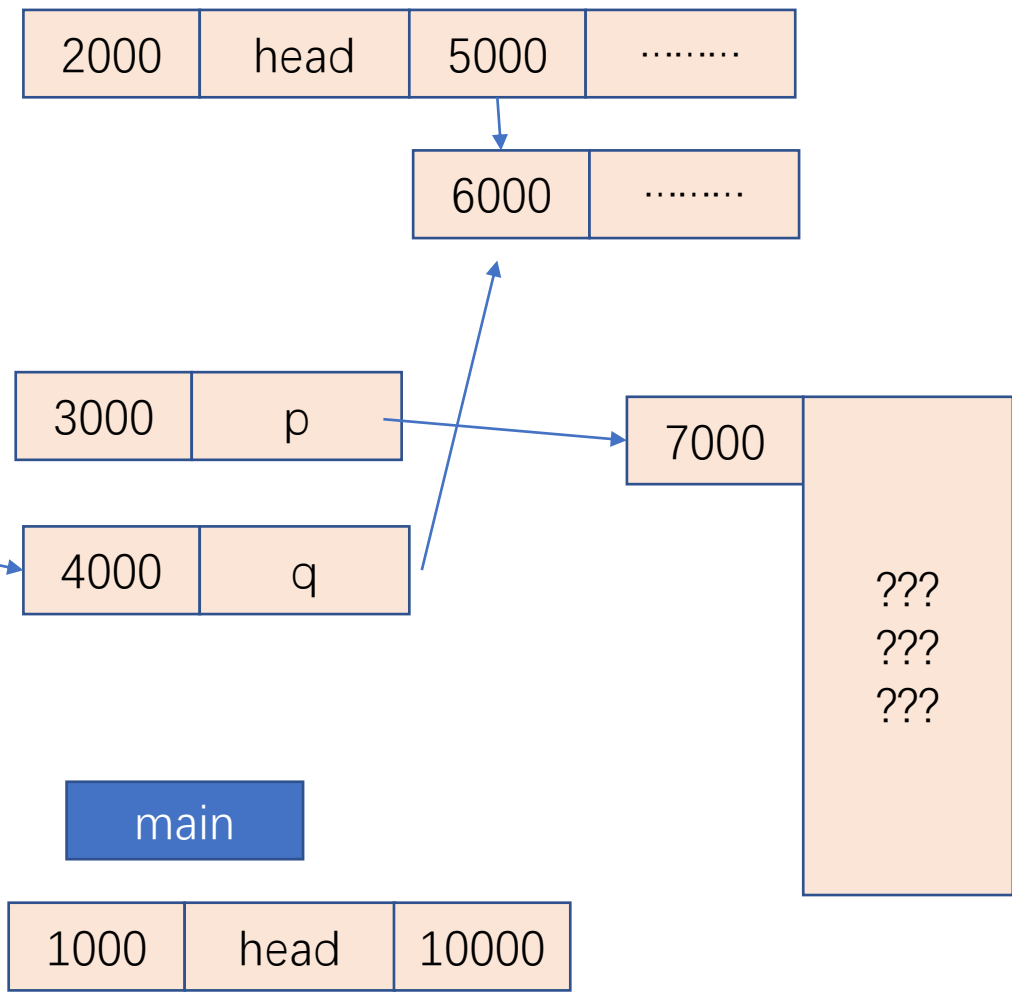


原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是后面的元素



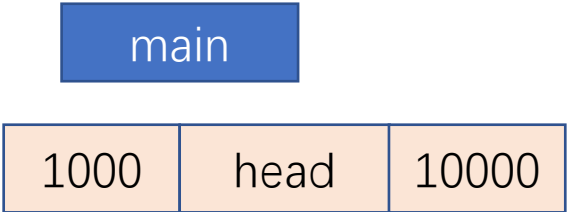
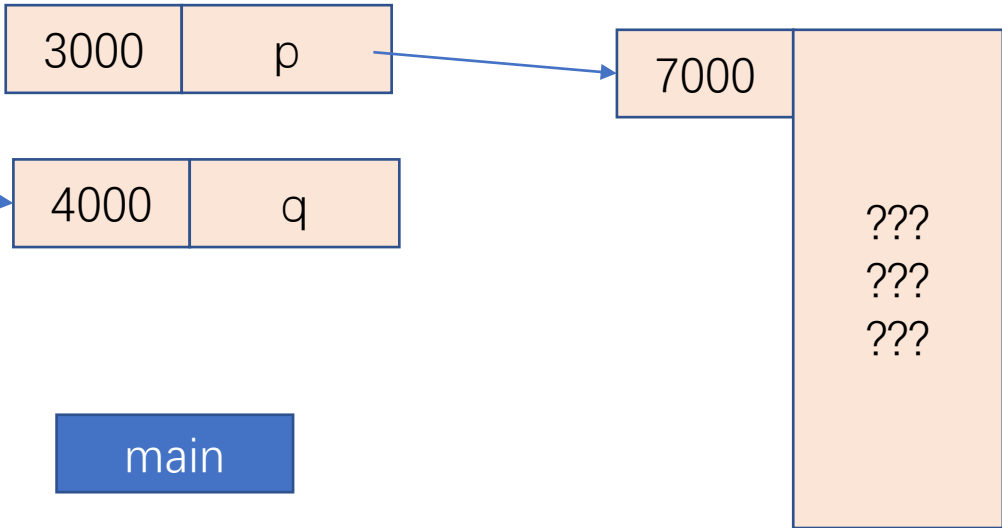
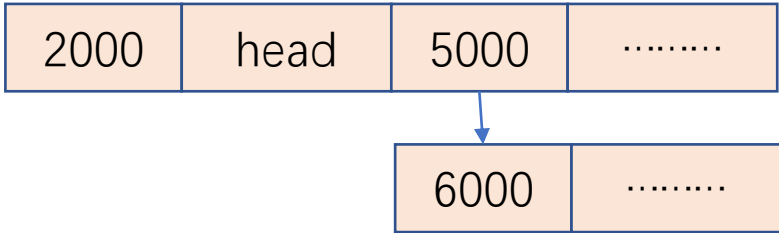
地址

原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是后面的元素



地址

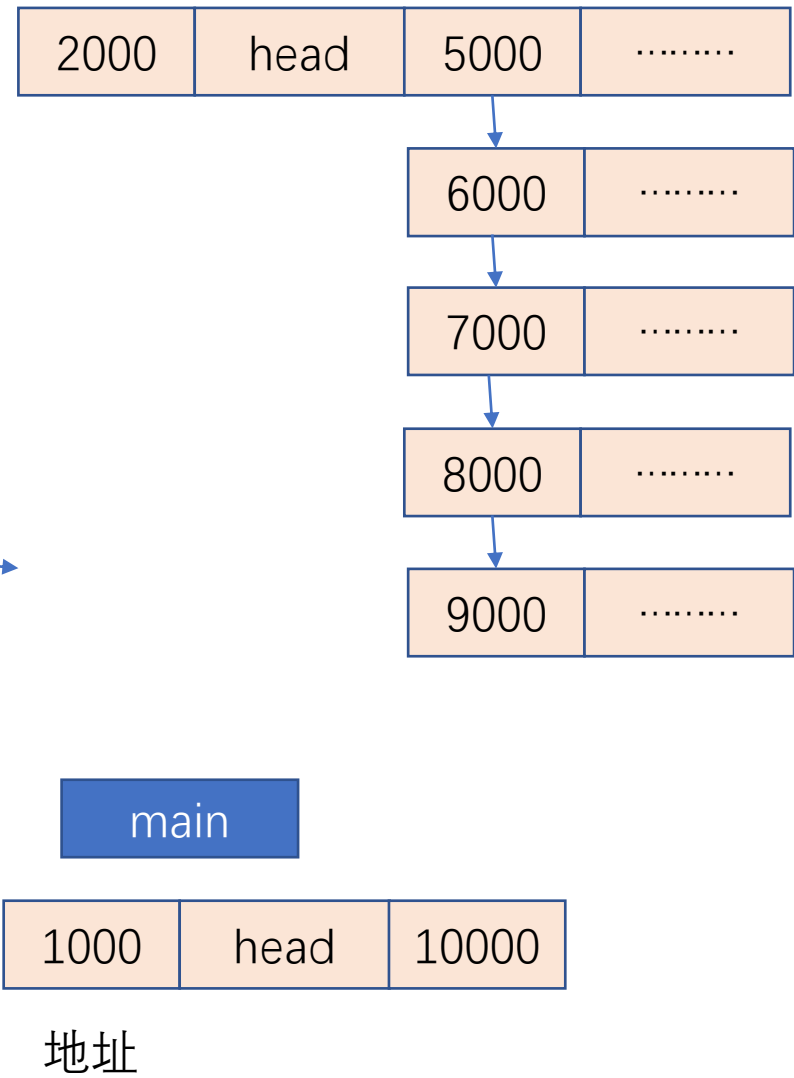
原先的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

Main中的head的值并没有得到修改

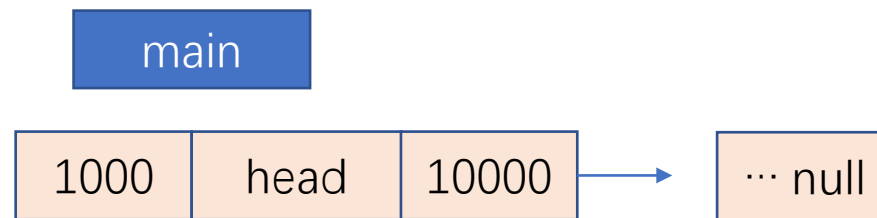
最后



遍历函数

```
int linklist_traverse(struct student* head)
{
    struct student* p;

    p = head; //p复位, 指向第1个结点
    while (p != NULL)
    { //循环进行输出
        printf("%s %d %c\n", p->name, p->num, p->sex);
        p = p->next;
    }
    return OK;
}
```



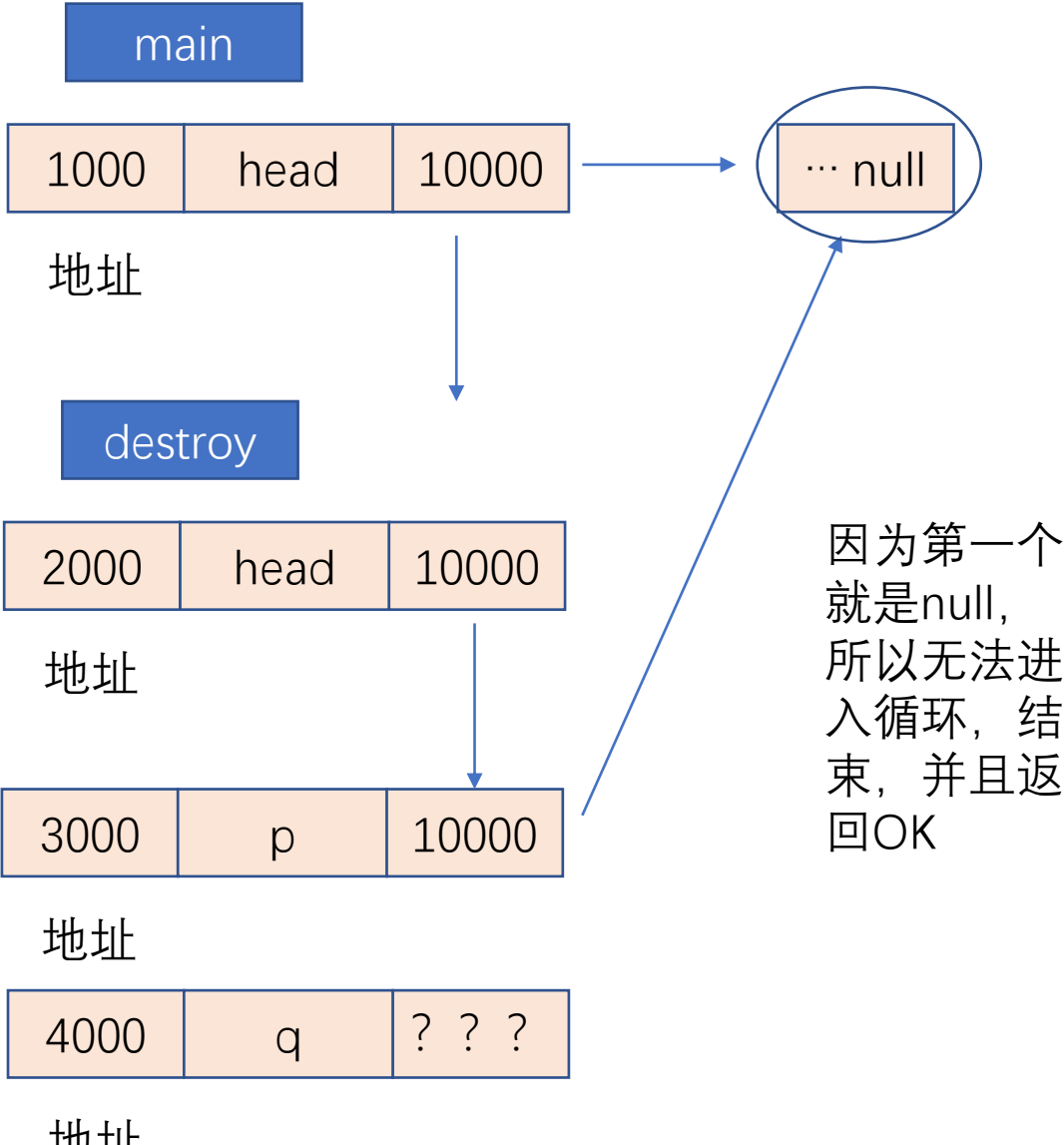
地址

由于main中head的值没有改变，
一直指向null，则根本没有任何遍
历就结束了

链表摧毁函数

```
int linklist_destroy(struct student* head)
{
    struct student* p, * q;

    p = head; //p复位, 指向第1个结点
    while (p)
    { //循环进行各结点释放
        q = p->next;
        free(p);
        p = q;
    }
    return OK;
}
```



参数修改

Create和声明

```
int linklist_create(struct student** head);
int linklist_traverse(struct student* head);
int linklist_destroy(struct student* head);

/*****
函数名称: linklist_create
功    能: 建立链表
输入参数: 构造体指针head
返 回 值: int
说    明:
*****/
int linklist_create(struct student** head)
```

Main函数

```
struct student* head = NULL;

if (linklist_create(&head) == OK)
{
    linklist_traverse(head);
    linklist_destroy(head);
}
else
```

destroy

2000	head	1000
------	------	------

地址

在17行, 原 `int linklist_create(struct student* head);`
现 `int linklist_create(struct student** head);`
在28行, 原 `int linklist_create(struct student* head)`
现 `int linklist_create(struct student** head)`

main

1000	head	10000
------	------	-------

地址

在103行, 原 `if (linklist_create(head) == OK)`
现 `if (linklist_create(&head) == OK)`

函数内部修改

```
int linklist_create(struct student** head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            *head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

在create中一直有main中head的地址, 这样通过head地址去修改main中head的值可以达到目标,而非之前的给一个head指向的地址, 在create一下子就丢了

此处会导致内存丢失, 因为一旦p为Null就退出了。但是之前申请成功的内存就会丢失

在41行, 原head = p;
现*head = p;

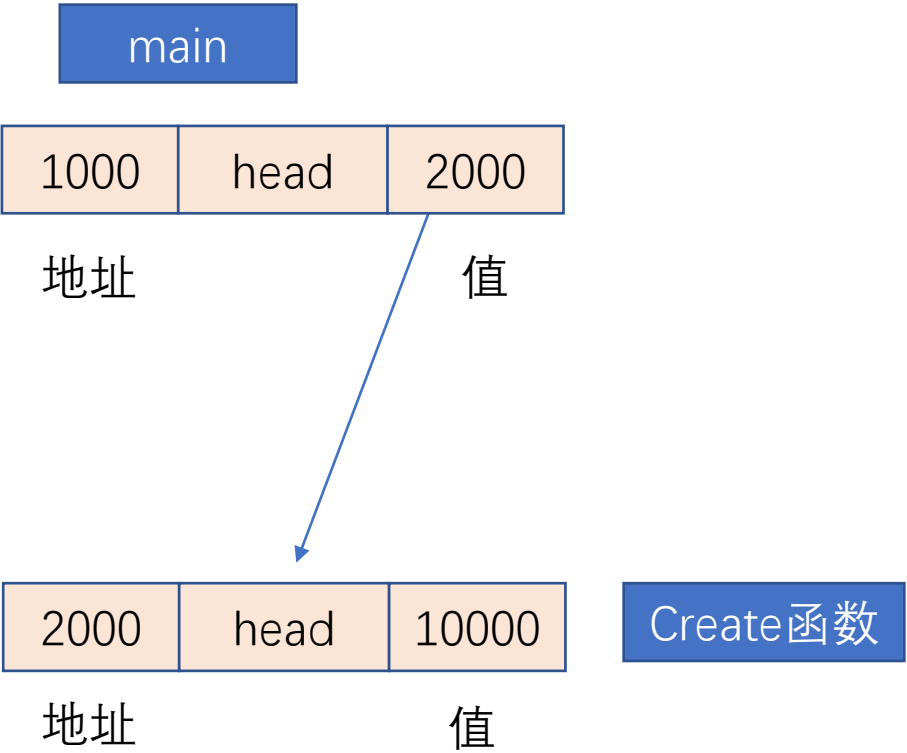
应该这么写

```
if (p == NULL)
{
    // 内存分配失败, 调用 linklist_destroy 释放已分配的节点
    linklist_destroy(*head);
    *head = NULL; // 设定头指针为 NULL
    return ERROR;
}
```

改后的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点,就借助操作系统来释放(非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

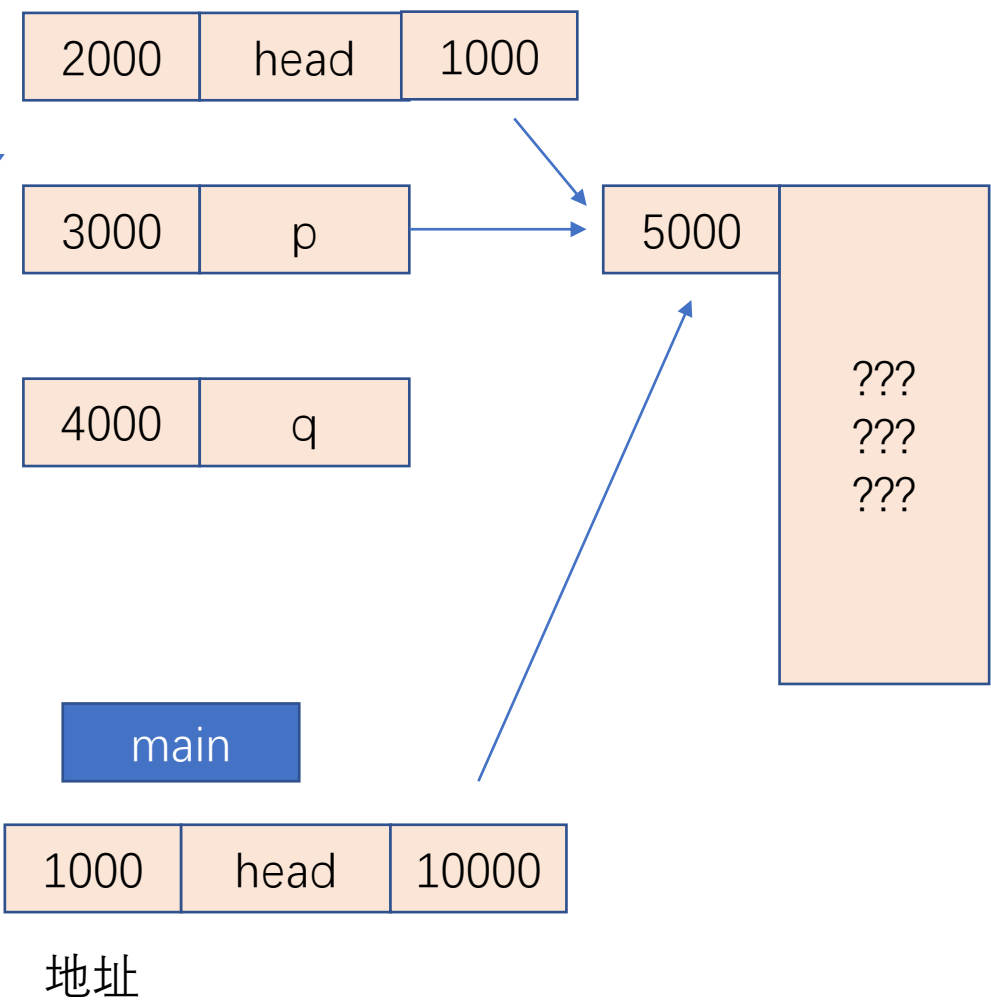


改后的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是第一个元素

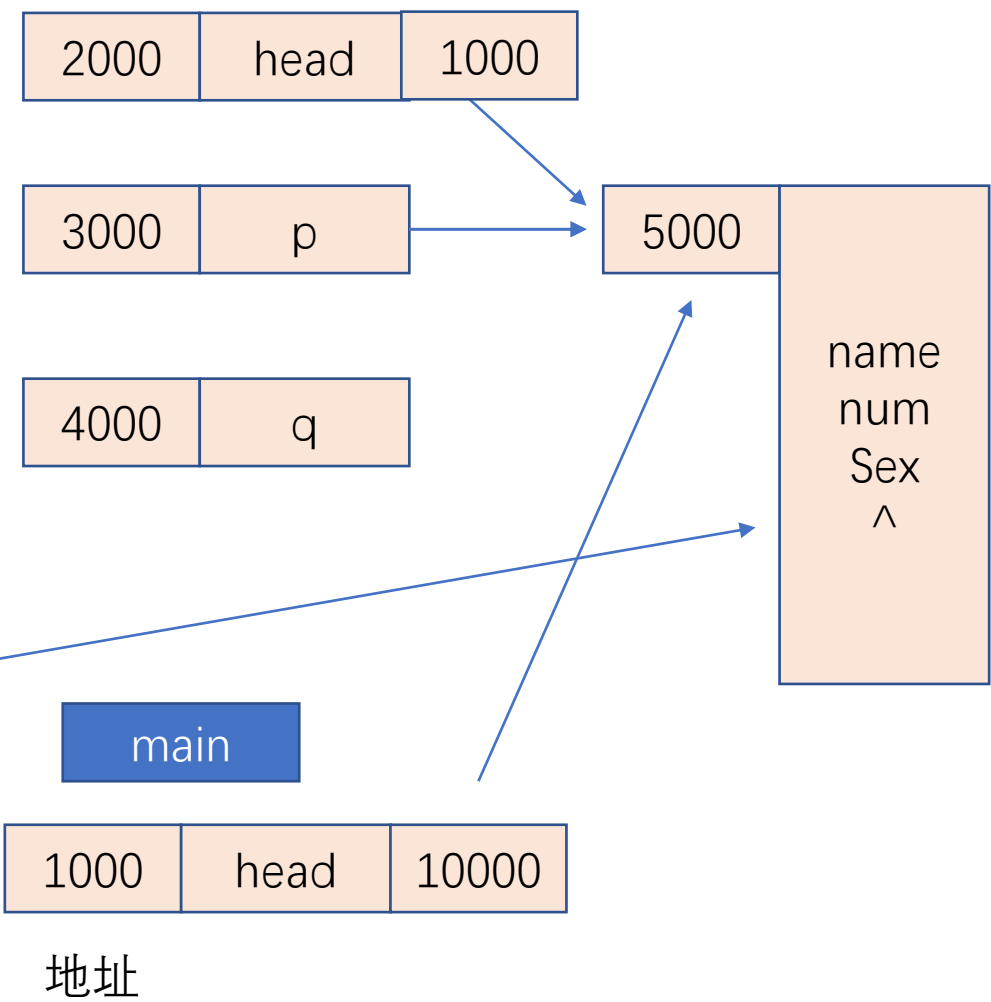


改后的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是第一个元素

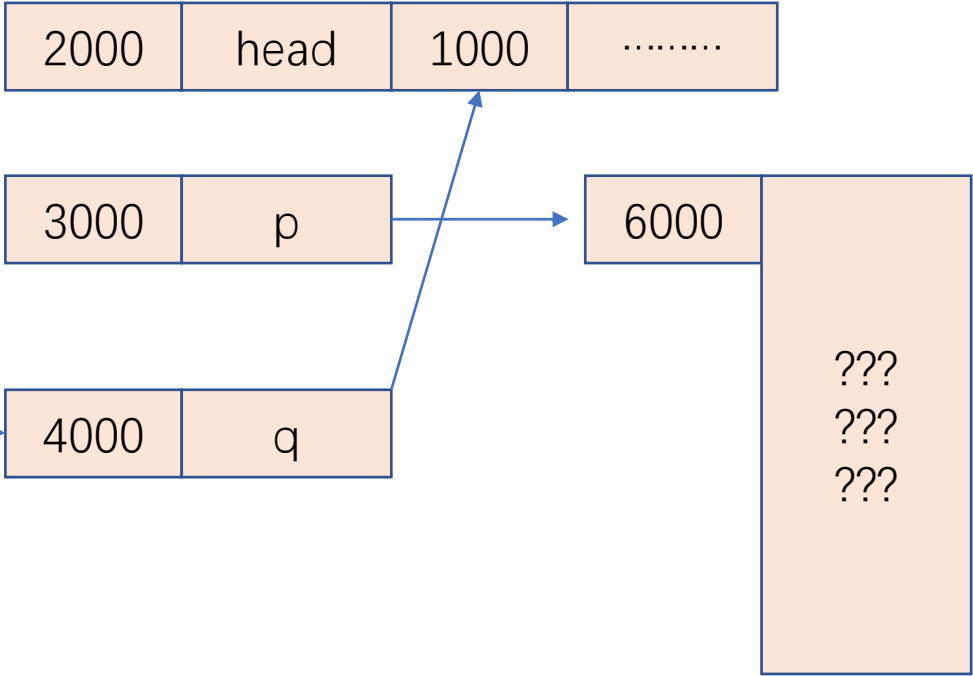


改后的create函数

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```

如果是后面的元素



main



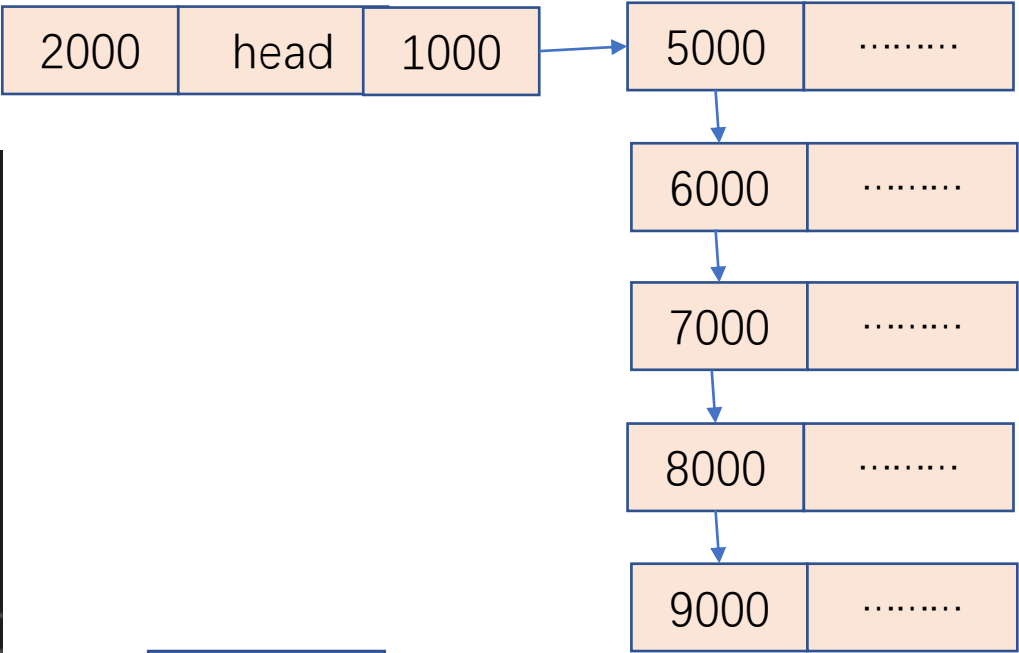
地址

改后的create函数

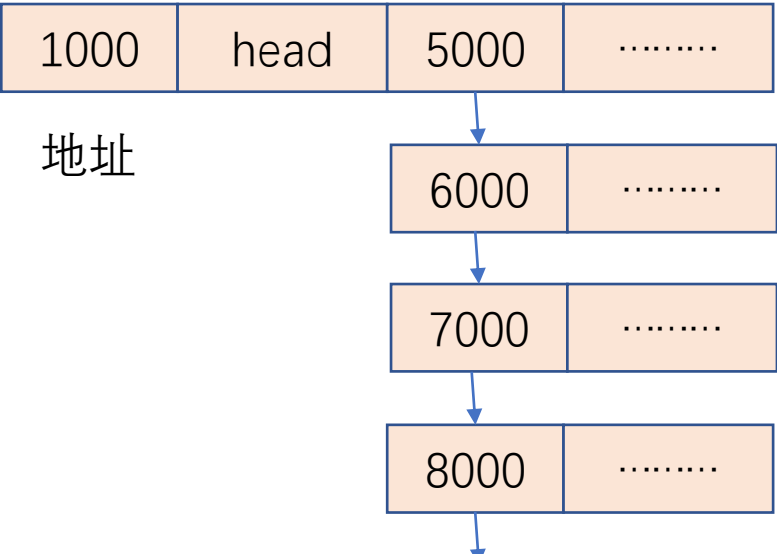
最后

```
int linklist_create(struct student* head)
{
    struct student* p = NULL, * q = NULL;
    int i;

    for (i = 0; i < 5; i++)
    {
        if (i > 0)
            q = p;
        p = (struct student*)malloc(sizeof(struct student));
        if (p == NULL)
            return ERROR; //注:此处未释放之前的链表节点, 就借助操作系统来释放 (非标用法)
        if (i == 0)
            head = p; //head指向第1个结点
        else
            q->next = p;
        printf("请输入第%d个人的基本信息\n", i + 1);
        scanf("%s %d %c", p->name, &(p->num), &(p->sex)); //键盘输入基本信息
        p->next = NULL;
    }
    return OK;
}
```



main



Main中的head的值并没有得到修改